

Name: Monisha Raghini Chelle

RollNo:2520030537

Section-6

OSSP (Operating Systems And Systems Programming) - 25CS2104E

(SKILL Content Delivery Concepts)

20.To Capture SIGTSTP, Suspend Processes, Update Job Table, Preserve State, Resume Later, Test Suspension Workflow.

To Create Process Groups, Assign Group IDs, Transfer Terminal Control, Restore Ownership, Coordinate Signals, Validate Behavior.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~$ ls
Linux-System-Programming-Week1  OSLAB14  Simple-Commands-Week1  foldername  hello.c  monis  sample1.c
OSLAB                           OSSP-Practical  example.txt             hello       moni    sample1  sample1.c.save
monis@DESKTOP-C3GM4T6:~$ cd OSLAB
monis@DESKTOP-C3GM4T6:~/OSLAB$ ls
OSLAB28  Scheduling-Algorithms  hello      hello.c:Zone.Identifier  hello_link  newfile.txt  test1
Pipes    file1.txt              hello.c    hello:Zone.Identifier    hello_shortcut  skill        virtual_address
monis@DESKTOP-C3GM4T6:~/OSLAB$ cd skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 2 — Create sigtstp_jobs.c

nano sigtstp_jobs.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano sigtstp_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 sigtstp_jobs.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sys/wait.h>

typedef enum { EMPTY, RUNNING, STOPPED, COMPLETED } State;

typedef struct {
    int id;
    pid_t pid;
    pid_t pgid;
    State state;
} Job;

static Job job = {0};
static volatile sig_atomic_t foreground_pgid = 0;

static void handle_sigtstp(int signo)
{
    (void)signo;

    if (foreground_pgid > 0)
        kill(-(pid_t)foreground_pgid, SIGTSTP);
}

static const char *state_name(State state)
{
    switch (state) {
        case RUNNING: return "RUNNING";
        case STOPPED: return "STOPPED";
        case COMPLETED: return "COMPLETED";
        default: return "EMPTY";
    }
}

static void update_job(void)
{
    if (job.state != RUNNING)
        return;
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 sigtstp_jobs.c *
int status;
pid_t result = waitpid(job.pid, &status,
                        WNOHANG | WUNTRACED);

if (result == job.pid) {
    if (WIFSTOPPED(status))
        job.state = STOPPED;
    else if (WIFEXITED(status) || WIFSIGNALED(status))
        job.state = COMPLETED;
}
}

static void show_job(void)
{
    update_job();

    printf("\nJOB TABLE\n");
    printf("ID\tPID\tPGID\tSTATE\n");

    if (job.state == EMPTY) {
        printf("No job records.\n");
        return;
    }

    printf("%d\t%d\t%d\t%s\n",
           job.id, (long)job.pid, (long)job.pgid,
           state_name(job.state));
}

static void wait_for_foreground(void)
{
    int status;
    pid_t result;

    foreground_pgid = (sig_atomic_t)job.pgid;

    do {
        result = waitpid(job.pid, &status, WUNTRACED);
    } while (result == -1 && errno == EINTR);

    foreground_pgid = 0;

    if (result == -1) {
        perror("waitpid");
        return;
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-F Redo
```

```
    }

    if (WIFSTOPPED(status)) {
        job.state = STOPPED;
        printf("\nJob [%d] suspended.\n", job.id);
    } else {
        job.state = COMPLETED;
        printf("\nJob [%d] completed.\n", job.id);
    }

    printf("Parent shell is still running.\n");
}

static void start_job(int seconds)
{
    update_job();

    if (job.state == RUNNING || job.state == STOPPED) {
        printf("Finish the existing job first.\n");
        return;
    }

    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        setpgid(0, 0);

        struct sigaction sa = {0};
        sa.sa_handler = SIG_DFL;
        sigemptyset(&sa.sa_mask);
        sigaction(SIGTSTP, &sa, NULL);

        char duration[20];
        snprintf(duration, sizeof(duration), "%d", seconds);

        execlp("sleep", "sleep", duration, (char *)NULL);
        perror("execlp");
        _exit(127);
    }
}
```

```
    if (setpgid(pid, pid) == -1 && errno != EACCES)
        perror("setpgid");

    job.id++;
    job.pid = pid;
    job.pgid = pid;
    job.state = RUNNING;

    printf("Started foreground job [%d], PID=%ld\n",
           job.id, (long)pid);
    printf("Press Ctrl+Z to suspend it.\n");
    fflush(stdout);

    wait_for_foreground();
}

static void resume_background(int id)
{
    update_job();

    if (job.id != id || job.state != STOPPED) {
        printf("Stopped job not found.\n");
        return;
    }

    if (kill(-job.pgid, SIGCONT) == -1) {
        perror("SIGCONT");
        return;
    }

    job.state = RUNNING;
    printf("Job [%d] resumed in background.\n", id);
}

static void resume_foreground(int id)
{
    update_job();

    if (job.id != id || job.state != STOPPED) {
        printf("Stopped job not found.\n");
        return;
    }

    if (kill(-job.pgid, SIGCONT) == -1) {
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 sigtstp_jobs.c *

    if (kill(-job.pgid, SIGCONT) == -1) {
        perror("SIGCONT");
        return;
    }

    job.state = RUNNING;
    printf("Job [%d] resumed in foreground.\n", id);
    fflush(stdout);

    wait_for_foreground();
}

int main(void)
{
    struct sigaction sa = {0};
    char input[100];
    int number;
    char extra;

    sa.sa_handler = handle_sigtstp;
    sigemptyset(&sa.sa_mask);

    if (sigaction(SIGTSTP, &sa, NULL) == -1) {
        perror("sigaction");
        return 1;
    }

    printf("Q20 - SIGTSTP and Job Suspension\n");
    printf("Commands: run NUMBER, jobs, bg ID, fg ID, exit\n");

    while (1) {
        update_job();

        printf("\nsuspend> ");
        fflush(stdout);

        if (!fgets(input, sizeof(input), stdin)) {
            if (ferror(stdin) && errno == EINTR) {
                clearerr(stdin);
                continue;
            }
            break;
        }
    }
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 sigtstp_jobs.c *
printf("Commands: run NUMBER, jobs, bg ID, fg ID, exit\n");

while (1) {
    update_job();

    printf("\nsuspend> ");
    fflush(stdout);

    if (!fgets(input, sizeof(input), stdin)) {
        if (ferror(stdin) && errno == EINTR) {
            clearerr(stdin);
            continue;
        }
        break;
    }

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;
    else if (strcmp(input, "jobs") == 0)
        show_job();
    else if (sscanf(input, "run %d %c",
                    &number, &extra) == 1 && number >= 0)
        start_job(number);
    else if (sscanf(input, "bg %d %c",
                    &number, &extra) == 1)
        resume_background(number);
    else if (sscanf(input, "fg %d %c",
                    &number, &extra) == 1)
        resume_foreground(number);
    else
        printf("Invalid command.\n");
}

if (job.state == STOPPED)
    kill(-job.pgid, SIGCONT);

if (job.state == RUNNING || job.state == STOPPED)
    waitpid(job.pid, NULL, 0);

printf("Suspension program exited.\n");
return 0;
}
```

Save using Ctrl + O, Enter, then Ctrl + X.

```
Write to File: sigtstp_jobs.c
^G Help      M-D DOS Format  M-A Append     M-B Backup File ^T Browse
^C Cancel    M-M Mac Format  M-P Prepend    ^O Discard buffer
```

Compile:

```
gcc -Wall -Wextra sigtstp_jobs.c -o sigtstp_jobs
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano sigtstp_jobs.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra sigtstp_jobs.c -o sigtstp_jobs
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 3 — Test job suspension and recovery

Run:

`./sigtstp_jobs`

Enter run 60,

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./sigtstp_jobs
Q20 - SIGTSTP and Job Suspension
Commands: run NUMBER, jobs, bg ID, fg ID, exit

suspend> 60
Invalid command.

suspend> run 60
Started foreground job [1], PID=1623
Press Ctrl+Z to suspend it.
```

then press **Ctrl + Z** while the job is running.

Resume the stopped job:

`bg 1`

`jobs`

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./sigtstp_jobs
Q20 - SIGTSTP and Job Suspension
Commands: run NUMBER, jobs, bg ID, fg ID, exit

suspend> 60
Invalid command.

suspend> run 60
Started foreground job [1], PID=1623
Press Ctrl+Z to suspend it.
^]^Z
Job [1] suspended.
Parent shell is still running.

suspend> jobs

JOB TABLE
ID      PID      PGID      STATE
1       1623     1623     STOPPED

suspend> bg 1
Job [1] resumed in background.

suspend> jobs

JOB TABLE
ID      PID      PGID      STATE
1       1623     1623     COMPLETED

suspend>
```

Step 4 — Create process_groups.c

Run:

nano process_groups.c

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano process_groups.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Paste the complete code:

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 process_groups.c *
#define _POSIX_C_SOURCE 200809L

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <errno.h>
#include <sys/wait.h>
#include <termios.h>

typedef enum { EMPTY, RUNNING, STOPPED, COMPLETED } State;

typedef struct {
    int id;
    pid_t pid;
    pid_t pgid;
    State state;
} Job;

static Job job = {0};
static pid_t shell_pgid;
static struct termios shell_termios;

static const char *state_name(State state)
{
    switch (state) {
        case RUNNING: return "RUNNING";
        case STOPPED: return "STOPPED";
        case COMPLETED: return "COMPLETED";
        default: return "EMPTY";
    }
}

static void update_job(void)
{
    if (job.state != RUNNING && job.state != STOPPED)
        return;

    int status;
    pid_t result;

    do {
        result = waitpid(job.pid, &status,
                        WNOHANG | WUNTRACED | WCONTINUED);
    } while (result == 0);
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
        if (result == job.pid) {
            if (WIFSTOPPED(status))
                job.state = STOPPED;
            else if (WIFCONTINUED(status))
                job.state = RUNNING;
            else if (WIFEXITED(status) || WIFSIGNALED(status))
                job.state = COMPLETED;
        }
    } while (result == job.pid &&
             job.state != COMPLETED);
}

static void show_job(void)
{
    update_job();

    printf("\nJOB TABLE\n");
    printf("ID\tPID\tPGID\tSTATE\n");

    if (job.state == EMPTY) {
        printf("No job records.\n");
        return;
    }

    printf("%d\t%d\t%d\t%s\n",
           job.id, (long)job.pid, (long)job.pgid,
           state_name(job.state));
}

static void restore_terminal(void)
{
    if (tcsetpgrp(STDIN_FILENO, shell_pgid) == -1)
        perror("restore tcsetpgrp");

    if (tcsetattr(STDIN_FILENO, TCSADRAIN,
                  &shell_termios) == -1)
        perror("restore terminal settings");
}

static void foreground_job(int resume)
{
    int status;
    pid_t result;

    if (tcsetpgrp(STDIN_FILENO, job.pgid) == -1) {
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```
if (tcsetpgrp(STDIN_FILENO, job.pgid) == -1) {
    perror("tcsetpgrp");
    return;
}

printf("Terminal transferred to PGID %ld.\n",
       (long)job.pgid);
fflush(stdout);

if (resume) {
    if (kill(-job.pgid, SIGCONT) == -1) {
        perror("SIGCONT");
        restore_terminal();
        return;
    }

    job.state = RUNNING;
    printf("Foreground job resumed.\n");
    fflush(stdout);
}

do {
    result = waitpid(job.pid, &status, WUNTRACED);
} while (result == -1 && errno == EINTR);

restore_terminal();

printf("\nTerminal restored to shell PGID %ld.\n",
       (long)shell_pgid);

if (result == -1) {
    perror("waitpid");
    return;
}

if (WIFSTOPPED(status)) {
    job.state = STOPPED;
    printf("Job [%d] stopped by signal %d.\n",
           job.id, WSTOPSIG(status));
}
else if (WIFSIGNALED(status)) {
    job.state = COMPLETED;
    printf("Job [%d] terminated by signal %d.\n",
           job.id, WTERMSIG(status));
}
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 process_groups.c *
}

static void start_job(int seconds)
{
    update_job();

    if (job.state == RUNNING || job.state == STOPPED) {
        printf("Finish the existing job first.\n");
        return;
    }

    pid_t pid = fork();

    if (pid == -1) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        if (setpgid(0, 0) == -1) {
            perror("setpgid");
            _exit(1);
        }

        signal(SIGINT, SIG_DFL);
        signal(SIGTSTP, SIG_DFL);
        signal(SIGTTIN, SIG_DFL);
        signal(SIGTTOU, SIG_DFL);

        char duration[20];
        snprintf(duration, sizeof(duration), "%d", seconds);

        execlp("sleep", "sleep", duration, (char *)NULL);
        perror("execlp");
        _exit(127);
    }

    if (setpgid(pid, pid) == -1 && errno != EACCES)
        perror("setpgid");

    job.id++;
    job.pid = pid;
    job.pgid = pid;
    job.state = RUNNING;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 process_groups.c *

printf("\nCreated job [%d].\n", job.id);
printf("Child PID: %ld\n", (long)job.pid);
printf("Child PGID: %ld\n", (long)job.pgid);
printf("Shell PGID: %ld\n", (long)shell_pgid);
printf("Press Ctrl+Z to suspend or Ctrl+C to interrupt.\n");
fflush(stdout);

foreground_job(0);
}

static void resume_background(int id)
{
    update_job();

    if (job.id != id || job.state != STOPPED) {
        printf("Stopped job not found.\n");
        return;
    }

    if (kill(-job.pgid, SIGCONT) == -1) {
        perror("SIGCONT");
        return;
    }

    job.state = RUNNING;

    printf("Job [%d] resumed in background.\n", id);
    printf("Terminal remains under shell control.\n");
}

int main(void)
{
    char input[100];
    int number;
    char extra;

    if (!isatty(STDIN_FILENO)) {
        fprintf(stderr, "Use an interactive terminal.\n");
        return 1;
    }

    shell_pgid = getpgrp();

    if (tcgetpgrp(STDIN_FILENO) != shell_pgid) {
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^_ Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
fprintf(stderr,
    "Start this program as a foreground terminal job.\n");
return 1;
}

if (tcgetattr(STDIN_FILENO, &shell_termios) == -1) {
    perror("tcgetattr");
    return 1;
}

/*
 * The shell must be able to reclaim the terminal
 * while it is temporarily in the background.
 */
signal(SIGTTOU, SIG_IGN);

printf("Q20 - Process Groups and Terminal Control\n");
printf("Shell PID: %ld\n", (long) getpid());
printf("Shell PGID: %ld\n", (long) shell_pgid);
printf("Commands: run NUMBER, jobs, fg ID, bg ID, exit\n");

while (1) {
    update_job();

    printf("\npgshell> ");
    fflush(stdout);

    if (!fgets(input, sizeof(input), stdin))
        break;

    input[strcspn(input, "\n")] = '\0';

    if (strcmp(input, "exit") == 0)
        break;
    else if (strcmp(input, "jobs") == 0)
        show_job();
    else if (sscanf(input, "run %d %c",
        &number, &extra) == 1 && number >= 0)
        start_job(number);
    else if (sscanf(input, "fg %d %c",
        &number, &extra) == 1) {
        update_job();

        if (job.id == number &&
            (job.state == RUNNING || job.state == STOPPED))
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^_ Replace

^K Cut
^U Paste

^T Execute
^J Justify

^C Location
^_ Go To Line

M-U Undo
M-E Redo

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
GNU nano 8.7.1 process_groups.c *

printf("\npgshell> ");
fflush(stdout);

if (!fgets(input, sizeof(input), stdin))
    break;

input[strcspn(input, "\n")] = '\0';

if (strcmp(input, "exit") == 0)
    break;
else if (strcmp(input, "jobs") == 0)
    show_job();
else if (sscanf(input, "run %d %c",
                &number, &extra) == 1 && number >= 0)
    start_job(number);
else if (sscanf(input, "fg %d %c",
                &number, &extra) == 1) {
    update_job();

    if (job.id == number &&
        (job.state == RUNNING || job.state == STOPPED))
        foreground_job(job.state == STOPPED);
    else
        printf("Active job not found.\n");
}
else if (sscanf(input, "bg %d %c",
                &number, &extra) == 1)
    resume_background(number);
else
    printf("Invalid command.\n");
}

restore_terminal();

if (job.state == STOPPED)
    kill(-job.pgid, SIGCONT);

if (job.state == RUNNING || job.state == STOPPED)
    waitpid(job.pid, NULL, 0);

printf("Process group program exited.\n");
return 0;
}

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line M-E Redo

Write to File: process_groups.c
^G Help      M-D DOS Format M-A Append    M-B Backup File ^T Browse
^C Cancel     M-M Mac Format M-P Prepend   ^O Discard buffer
```

Save and compile:

```
gcc -Wall -Wextra process_groups.c -o process_groups
```

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ nano process_groups.c
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ gcc -Wall -Wextra process_groups.c -o process_groups
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$
```

Step 5 — Test process groups and terminal control

Run:

`./process_groups`

Enter run 60 and immediately press Ctrl + Z.

```
monis@DESKTOP-C3GM4T6: ~/OSLAB/skill
monis@DESKTOP-C3GM4T6:~/OSLAB/skill$ ./process_groups
Q20 - Process Groups and Terminal Control
Shell PID: 1805
Shell PGID: 1805
Commands: run NUMBER, jobs, fg ID, bg ID, exit

pgshell> run 60

Created job [1].
Child PID: 1806
Child PGID: 1806
Shell PGID: 1805
Press Ctrl+Z to suspend or Ctrl+C to interrupt.
Terminal transferred to PGID 1806.
^]^Z
Terminal restored to shell PGID 1805.
Job [1] stopped by signal 20.

pgshell> exit
```

